دليل الأسئلة للطلاب

دالة صغيرة.
وفكرة واضحة.

Guide شيت 5: اقرأ السؤال الأصلي، افهم الفكرة، ثم تابع الكود والنتيجة. كل سؤال له مثال متتبع وأخطاء شائعة وحالات تشغيل حقيقية.

الشرح: أحمد طولان · Teaching assistant
المصادر الأصلية: Prof. Mahmoud M. Khater · 2026/2027

الشيت الأصلي Lecture 5 — التكرار والأساسيات دليل Python قبل الحل

Lecture 5 مرجع للمتطلبات السابقة: f-strings والتكرار وrange والأعداد العشوائية. شرح functions هنا مستند إلى أكواد شيت 5 وتوثيق Python؛ لا نعرضها على أنها موضوع Lecture 5.

01 التكعيب: أول function ترجع نتيجة

(1) Write a Python program in which the user is promoted to enter a number and find its cube using function.

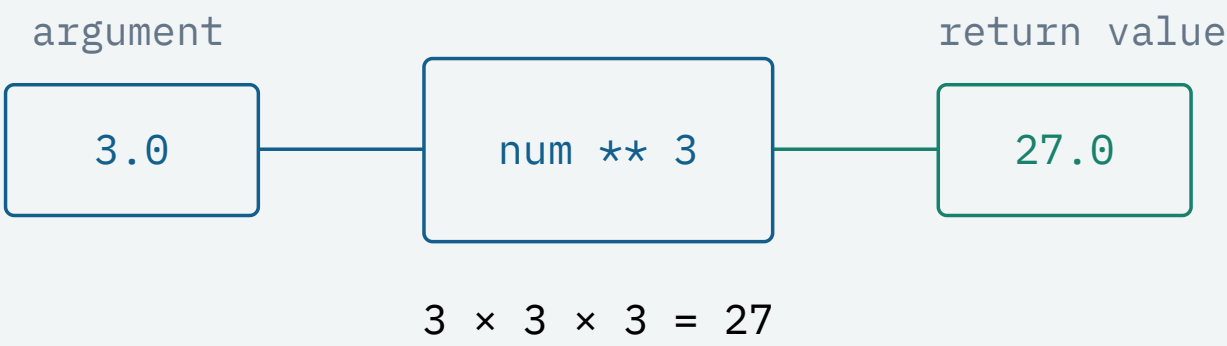
نص الدكتور الأصلي — اضغط الصورة للتكبير.

المطلوب إيه؟

المستخدم يدخل عددًا، والدالة تحسب مكعبه، والبرنامج يستلم الناتج ويطبعه. كلمة cube معناها العدد مضروب في نفسه ثلاث مرات، مش مضروب في 3.

الفكرة قبل الكود

افصل بين تعريف طريقة الحساب واستعمالها. Python يقابل def ويسجل الدالة، لكن جسمها لا يعمل إلا عند الاستدعاء. الاسم num داخل الدالة يستقبل قيمة number؛ الاسمان مش لازم يكونوا متطابقين.



نتتبع مثالًا

للمدخل 3: number = 3.0؛ الاستدعاء calculate_cube(number) يجعل num = 3.0؛ math.pow ينتج 27.0؛ return يسلّم 27.0 إلى result؛ ثم print يعرضها.

Python اللي السؤال محتاجها

input → float → def والـindentation → parameter وargument → return → استدعاء داخل إسناد → f-string.

الحل، سطرًا بسطر

اقرأ تعريف الدالة أولًا، ثم ابدأ التنفيذ من البرنامج الرئيسي. عند الاستدعاء ارجع لجسم الدالة، وعند return عد لمكان الاستدعاء.

```python
# Sheet_05, Q_01
import math
# Function to calculate the cube of a number
def calculate_cube(num):
    result = math.pow(num, 3)
    return result

# Main program
number = float(input("Enter a number to find its cube: "))
result = calculate_cube(number)

# Display the result
print(f"The cube of {number} is: {result}")
```

▼ شرح السطور واحدة واحدة

1. `# Sheet_05, Q_01`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

2. `import math`

   أتح دوال المكتبة .math نستخدم اسم المكتبة ثم نقطة ثم اسم الدالة.

3. `# Function to calculate the cube of a number`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

4. `def calculate_cube(num):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

5. `result = math.pow(num, 3)`

   احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

6. `return result`

   أعد قيمة result إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

7. `# Main program`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

8. `number = float(input("Enter a number to find its cube: "))`

   اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (floatأو int)، وخزّن النتيجة في الاسم على اليسار.

9. `result = calculate_cube(number)`

   احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

10. `# Display the result`

    تعليق يشرح الغرض؛ Python لا ينفّذه.

11. `print(f"The cube of {number} is: {result}")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

تشغيل فعلي

المدخلات، ثم stdout كما سجّله Python. رسائل input قد تظهر على نفس السطر لأن التسجيل لا يكرر ما كتبته لوحة المفاتيح.

المدخلات

```
3
```

stdout

```
Enter a number to find its cube: The cube of 3.0 is: 27.0
```

نصائح وأخطاء شائعة

- في المرجع math.pow(num, 3) تحسب القوة وتنتج float. كتابة num ** 3 بديل مباشر، و^ ليس عامل الأس في Python.
- return يرجّع قيمة إلى مكان الاستدعاء؛ print يعرض على الشاشة. لا تستبدل أحدهما بالآخر.
- ارجع لمكان الاستدعاء بعد return: نتيجة الدالة هي التي تُسند إلى result.

اختبر فهمك

لو كتبنا تعريف الدالة فقط، هل تظهر نتيجة؟

▼ اعرض السبب

لا. التعريف يسجّل الدالة، والاستدعاء هو الذي ينفّذ جسمها.

لما تقارن بالمرجع: الشرح يحافظ على math.pow وfloat وf-string كما في كود الدكتور. كلمة promoted في نص الشيت مقصود بها prompted: يُطلب من المستخدم الإدخال.

02 ثلاث عمليات، وثلاث دوال

(2) Write a Python program, which prompt the user to enter two numbers and use functions to add, subtract and multiply these numbers.

نص الدكتور الأصلي — اضغط الصورة للتكبير.

المطلوب إيه؟

اقرأ عددين مرة واحدة، واستدعِ دالة للجمع ودالة للطرح ودالة للضرب، واعرض النتائج الثلاث.

الفكرة قبل الكود

كل دالة لها مهمة محددة وتستقبل قيمتين بالترتيب. نفس القيم تدخل ثلاث عمليات مختلفة. في subtract(num1, num2) نحسب الأول ناقص الثاني؛ الترتيب يغيّر نتيجة الطرح.

```
add(8, 3)          subtract(8, 3)          multiply(8, 3)
8 + 3 = 11         8 - 3 = 5               8 × 3 = 24

subtract(3, 8) = -5
```

نتتبع مثالًا

عند إدخال 8 ثم 3: add ترجع 11.0؛ subtract ترجع 5.0؛ multiply ترجع 24.0. استدعاء subtract(3, 8) ينتج -5.

Python اللي السؤال محتاجها

عامل + و- و*؛ دالة بوسيطين؛ ترتيب arguments؛ استدعاء متداخل داخل print.

الحل، سطرًا بسطر

اقرأ تعريف الدالة أولًا، ثم ابدأ التنفيذ من البرنامج الرئيسي. عند الاستدعاء ارجع لجسم الدالة، وعند return عد لمكان الاستدعاء.

```python
# Sheet_05, Q_02
# Function to add two numbers
def add(num1, num2):
    return num1 + num2

# Function to subtract two numbers
def subtract(num1, num2):
    return num1 - num2

# Function to multiply two numbers
def multiply(num1, num2):
    return num1 * num2

# Main program
number1 = float(input("Enter the first number: "))
number2 = float(input("Enter the second number: "))

# Perform operations using functions and display results
print("Sum:", add(number1, number2))
print("Difference:", subtract(number1, number2))
print("Product:", multiply(number1, number2))
```

▼ شرح السطور واحدة واحدة

1. `# Sheet_05, Q_02`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

2. `# Function to add two numbers`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

3. `def add(num1, num2):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

4. `return num1 + num2`

   أعد قيمة num1 + num2 إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

5. `# Function to subtract two numbers`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

6. `def subtract(num1, num2):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

7. `return num1 - num2`

   أعد قيمة num1 - num2 إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

8. `# Function to multiply two numbers`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

9. `def multiply(num1, num2):`

تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

10. `return num1 * num2`

أعد قيمة num1 * num2 إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

11. `# Main program`

تعليق يشرح الغرض؛ Python لا ينفّذه.

12. `number1 = float(input("Enter the first number: "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (int أوfloat)، وخزّن النتيجة في الاسم على اليسار.

13. `number2 = float(input("Enter the second number: "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (int أوfloat)، وخزّن النتيجة في الاسم على اليسار.

14. `# Perform operations using functions and display results`

تعليق يشرح الغرض؛ Python لا ينفّذه.

15. `print("Sum:", add(number1, number2))`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

16. `print("Difference:", subtract(number1, number2))`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

17. `print("Product:", multiply(number1, number2))`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

تشغيل فعلي

المدخلات، ثم stdout كما سجّله .Python رسائل input قد تظهر على نفس السطر لأن التسجيل لا يكرر ما كتبته لوحة المفاتيح.

المدخلات

```
8
3
```

stdout

```
Enter the first number: Enter the second number: Sum: 11.0
Difference: 5.0
Product: 24.0
```

نصائح وأخطاء شائعة

- لا تكرر input داخل كل دالة؛ السؤال يحتاج نفس العددين في العمليات الثلاث.
- Python يحسب add(number1, number2) أولًا، ثم يرسل الناتج إلى print.
- جرّب تبديل العددين: الجمع والضرب يبقيان نفس الناتج، والطرح يغيّر إشارته.

اختبر فهمك

هل subtract(8, 3) هي subtract(3, 8)؟

▼ اعرض السبب

لا؛ الأولى 5 والثانية -5. argument الأول يصل إلى parameter الأول.

لما تقارن بالمرجع: النسخة التعليمية مطابقة لطريقة المرجع في فصل العمليات إلى ثلاث دوال.

03 التبديل: احفظ القيمة ثم رجّع القيمتين

(3) Write a Python program, which prompt the user to enter two numbers and use a function to swap the values the two numbers.

نص الدكتور الأصلي — اضغط الصورة للتكبير.

المطلوب إيه؟

بعد التنفيذ، المتغير الأول يحمل القيمة القديمة للثاني، والثاني يحمل القيمة القديمة للأول. المطلوب تبديل القيمتين باستخدام دالة.

الفكرة قبل الكود

لو نفّذت num1 = num2 مباشرة فقدت قيمة num1 القديمة. المتغير temp يحتفظ بها أثناء التبديل. الأسماء داخل الدالة محلية؛ تغييرها وحده لا يغيّر أسماء البرنامج الرئيسي.

الخطوة	main: number1, number2	local: num1	local: num2	temp
دخول الدالة	8, 3	8	3	—
temp = num1	8, 3	8	3	8
num1 = num2	8, 3	3	3	8
num2 = temp	8, 3	3	8	8
استلام return في main	3, 8	انتهى الاستدعاء	—	—

نتتبع مثالًا

ابدأ بـ num1 = 8 و num2 = 3: temp يصبح 8؛ num1 يصبح 3؛ num2 يسترد 8 من temp؛ ترجع (3, 8)، ويستلمها الاسمان في البرنامج الرئيسي.

Python اللي السؤال محتاجها

الإسناد ينقل قيمة من اليمين لليسار؛ متغير مؤقت؛ نطاق محلي؛ return لقيمتين؛ tuple unpacking.

الحل، سطرًا بسطر

اقرأ تعريف الدالة أولًا، ثم ابدأ التنفيذ من البرنامج الرئيسي. عند الاستدعاء ارجع لجسم الدالة، وعند return عد لمكان الاستدعاء.

```python
# Sheet_05, Q_03
# Function to swap two integers
def swap_numbers(num1, num2):
    temp = num1
    num1 = num2
    num2 = temp
    return num1, num2

# Main program
number1 = int(input("Enter the first number: "))
number2 = int(input("Enter the second number: "))

# Display numbers before swapping
print("\nBefore swapping:")
print(f"First number: {number1} and Second number: {number2}")

# Call the swap_numbers function
number1, number2 = swap_numbers(number1, number2)

# Display numbers after swapping
print("\nAfter swapping:")
print(f"First number: {number1} and Second number: {number2}")
```

▼ شرح السطور واحدة واحدة

1. `# Sheet_05, Q_03`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

2. `# Function to swap two integers`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

3. `def swap_numbers(num1, num2):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

4. `temp = num1`

   احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

5. `num1 = num2`

   احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

6. `num2 = temp`

   احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

7. `return num1, num2`

   أعد قيمة num1, num2 إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

8. `# Main program`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

9. `number1 = int(input("Enter the first number: "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (int أوfloat)، وخزّن النتيجة في الاسم على اليسار.

10. `number2 = int(input("Enter the second number: "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (int أوfloat)، وخزّن النتيجة في الاسم على اليسار.

11. `# Display numbers before swapping`

تعليق يشرح الغرض؛ Python لا ينفّذه.

12. `print("\nBefore swapping:")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

13. `print(f"First number: {number1} and Second number: {number2}")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

14. `# Call the swap_numbers function`

تعليق يشرح الغرض؛ Python لا ينفّذه.

15. `number1, number2 = swap_numbers(number1, number2)`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر return.

16. `# Display numbers after swapping`

تعليق يشرح الغرض؛ Python لا ينفّذه.

17. `print("\nAfter swapping:")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

18. `print(f"First number: {number1} and Second number: {number2}")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

تشغيل فعلي

المدخلات، ثم stdout كما سجّله Python. رسائل input قد تظهر على نفس السطر لأن التسجيل لا يكرر ما كتبته لوحة المفاتيح.

المدخلات

```
8
3
```

stdout

```
Enter the first number: Enter the second number:
Before swapping:
First number: 8 and Second number: 3

After swapping:
First number: 3 and Second number: 8
```

نصائح وأخطاء شائعة

- الخطوة الحاسمة خارج الدالة هي number2 = swap_numbers(number1, number2). من غير الإسناد ستُهمل القيمتان الراجعتان.
- return num1, num2 يرجع tuple واحدًا يحتوي قيمتين، ثم نفكّه إلى اسمين.
- الاختصار num1, num2 = num2, num1 يحسب الطرف اليمين أولًا؛ افهم temp قبل استخدامه.

اختبر فهمك

لو استدعينا swap_numbers(number1, number2) من غير استلام الناتج، هل يتبدلان في البرنامج الرئيسي؟

▼ اعرض السبب

لا؛ إعادة إسناد الأسماء المحلية لا تعيد إسناد أسماء المستدعي.

لما تقارن بالمرجع: المرجع يستخدم int للعددين؛ أبقيناه لتسهيل المقارنة. نفس فكرة الدالة تعمل مع قيم عددية أخرى إذا تغيرت قراءة المدخلات.

04 هل العدد أولي؟ ابحث عن قاسم واحد

(4) Write a Python program that prompts the user to enter an integer and use a function, based on the *trial division method*, to check whether it is prime or not.

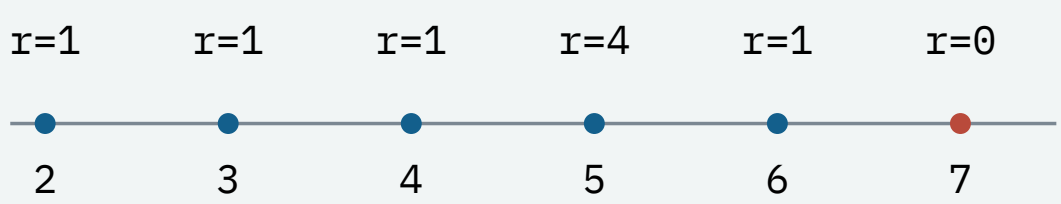
نص الدكتور الأصلي — اضغط الصورة للتكبير.

المطلوب إيه؟

اقرأ عددًا صحيحًا، وقرّر هل هو أكبر من 1 وله قاسمان موجبان فقط: 1 ونفسه. استخدم تجربة القسمة داخل دالة.

الفكرة قبل الكود

يكفي العثور على قاسم واحد غير 1 والعدد نفسه لنقول إنه ليس أوليًا. لو العدد مركب، يوجد عامل لا يتجاوز الجذر التربيعي؛ لأن عاملين أكبر من الجذر حاصل ضربهما أكبر من العدد.



نتتبع مثالًا

49: نجرّب 2 ثم 3 ثم 4 ثم 5 ثم 6 ثم 7. البواقي 1، 1، 1، 4، 1، 0. عند 7 نرجع .False أما 29 فنجرّب 2 إلى 5، ولا نجد قاسمًا، فنرجع .True

Python اللي السؤال محتاجها

قيمة Boolean؛ if؛ <=؛ %؛ ==؛ for وrange؛ math.sqrt؛ int؛ return مبكر.

الحل، سطرًا بسطر

اقرأ تعريف الدالة أولًا، ثم ابدأ التنفيذ من البرنامج الرئيسي. عند الاستدعاء ارجع لجسم الدالة، وعند return عد لمكان الاستدعاء.

```python
# Sheet_05, Q_04
import math
# Function to check if a number is prime
def is_prime(num):
    if num <= 1:
        return False

    # Check for divisibility from 2 up to the square root of num
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return False

    return True

# Main program
number = int(input("Enter an integer: "))
if is_prime(number):
    print(f"{number} is a prime number")
else:
    print(f"{number} is not a prime number")
```

▼ شرح السطور واحدة واحدة

1. `# Sheet_05, Q_04`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

2. `import math`

   أتح دوال المكتبة .math نستخدم اسم المكتبة ثم نقطة ثم اسم الدالة.

3. `# Function to check if a number is prime`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

4. `def is_prime(num):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

5. `if num <= 1:`

   اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة .True

6. `return False`

   أعد قيمة False إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

7. `# Check for divisibility from 2 up to the square root of num`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

8. `for i in range(2, int(math.sqrt(num)) + 1):`

   كرّر الجسم المزاح للداخل لكل قيمة في range بالترتيب؛ قيمة stop نفسها لا تدخل.

9. `if num % i == 0:`

   اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة .True

10. `return False`

أعد قيمة False إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

11. `return True`

أعد قيمة True إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

12. `# Main program`

تعليق يشرح الغرض؛ Python لا ينفّذه.

13. `number = int(input("Enter an integer: "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (int أوfloat)، وخزّن النتيجة في الاسم على اليسار.

14. `if is_prime(number):`

اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة .True

15. `print(f"{number} is a prime number")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

16. `else:`

هذا الفرع يعمل عندما لا يتحقق شرط if المقابل.

17. `print(f"{number} is not a prime number")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

تشغيل فعلي

المدخلات، ثم stdout كما سجّله .Python رسائل input قد تظهر على نفس السطر لأن التسجيل لا يكرر ما كتبته لوحة المفاتيح.

المدخلات

```
49
```

stdout

```
Enter an integer: 49 is not a prime number
```

نصائح وأخطاء شائعة

- ارفض num <= 1 قبل .sqrt الصفر و1 والأعداد السالبة ليست أولية.
- نهاية range لا تدخل في التكرار، لذلك نضيف 1 بعد int(math.sqrt(num)). العدد 49 يكشف نسيان هذه الزيادة.
- return True يأتي بعد الحلقة؛ فشل القسمة على 2 وحده لا يثبت أن 9 أولي.
- لأعداد صحيحة ضخمة جدًا يمكن استخدام math.isqrt؛ الأمثلة هنا أعداد تعليمية صغيرة.

اختبر فهمك

هل يكفي أن 9 لا يقبل القسمة على 2 لنرجع True؟

▼ اعرض السبب

لا. لازم نكمل المرشحين؛ 9 يقبل القسمة على 3.

لما تقارن بالمرجع: طريقة trial division وحدود الحلقة كما في المرجع. التفريق بين تعريف العدد الأولي واختبار القسمة جزء من الشرح.

05 الأعداد الأولية في مجال: إعادة استخدام الاختبار

(5) Write a Python program to find the prime integers between a lower and upper value. Prompt the user to enter these values and use a function to find the prime number.

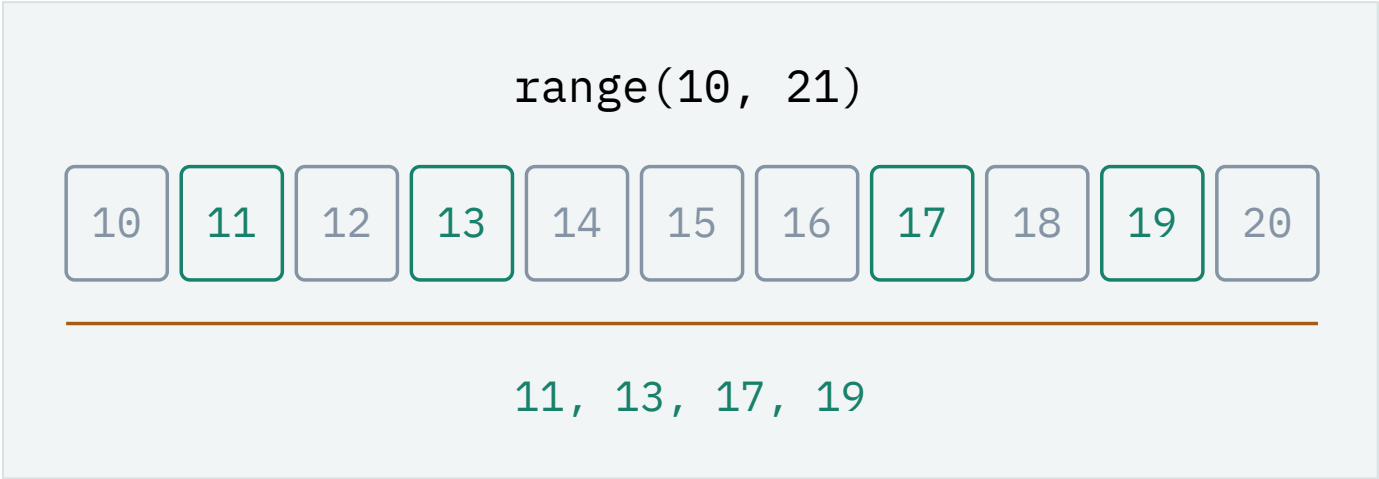
نص الدكتور الأصلي — اضغط الصورة للتكبير.

المطلوب إيه؟

اقرأ حدًا أدنى وحدًا أعلى. مرّ على كل عدد بينهما، واعرضه فقط إذا أثبتت is_prime أنه أولي. الطرفان داخلان، وفق تنفيذ المرجع.

الفكرة قبل الكود

لدينا مستويان مختلفان: حلقة خارجية تختار العدد المرشح، ودالة تختبر قواسم هذا العدد. الدالة لا تحفظ حكم العدد السابق؛ كل استدعاء له قيمه المحلية.



نتتبع مثالًا

المجال 10 إلى 20: المرشحون 10، 11، 12، 13، 14، 15، 16، 17، 18، 19، 20. المقبولون 11، 13، 17، 19. في المجال 2 إلى 2 يجب طباعة 2.

Python اللي السؤال محتاجها

استدعاء داخل شرط؛ range(lower, upper + 1)؛ print مع end؛ الفصل بين المرشح والقاسم.

الحل، سطرًا بسطر

اقرأ تعريف الدالة أولًا، ثم ابدأ التنفيذ من البرنامج الرئيسي. عند الاستدعاء ارجع لجسم الدالة، وعند return عد لمكان الاستدعاء.

```python
# Sheet_05, Q_05
import math
# Function to check if a number is prime
def is_prime(num):
    if num <= 1:
        return False
    if num == 2:
        return True
    if num % 2 == 0:
        return False

    for i in range(3, int(math.sqrt(num)) + 1, 2):
        if num % i == 0:
            return False

    return True

# Main program
lower_value = int(input("Enter the lower value of the range: "))
upper_value = int(input("Enter the upper value of the range: "))

if lower_value > upper_value:
    print("Error: Lower value cannot be greater than upper value.")
else:
    print(f"Prime numbers between {lower_value} and {upper_value} are: ", end="")

    for i in range(lower_value, upper_value + 1):
        if is_prime(i):
            print(i, end=" ")

    print()
```

▼ شرح السطور واحدة واحدة

1. `# Sheet_05, Q_05`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

2. `import math`

   أتح دوال المكتبة .math نستخدم اسم المكتبة ثم نقطة ثم اسم الدالة.

3. `# Function to check if a number is prime`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

4. `def is_prime(num):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

5. `if num <= 1:`

   اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة .True

6. `return False`

   أعد قيمة False إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

7. `if num == 2:`

اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة True.

8. `return True`

أعد قيمة True إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

9. `if num % 2 == 0:`

اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة True.

10. `return False`

أعد قيمة False إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

11. `for i in range(3, int(math.sqrt(num)) + 1, 2):`

كرّر الجسم المزاح للداخل لكل قيمة في range بالترتيب؛ قيمة stop نفسها لا تدخل.

12. `if num % i == 0:`

اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة True.

13. `return False`

أعد قيمة False إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

14. `return True`

أعد قيمة True إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

15. `# Main program`

تعليق يشرح الغرض؛ Python لا ينفّذه.

16. `lower_value = int(input("Enter the lower value of the range: "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (intأوfloat)، وخزّن النتيجة في الاسم على اليسار.

17. `upper_value = int(input("Enter the upper value of the range: "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (intأوfloat)، وخزّن النتيجة في الاسم على اليسار.

18. `if lower_value > upper_value:`

اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة True.

19. `print("Error: Lower value cannot be greater than upper value.")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

20. `else:`

هذا الفرع يعمل عندما لا يتحقق شرط if المقابل.

21. `print(f"Prime numbers between {lower_value} and {upper_value} are: ", end="")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

22. `for i in range(lower_value, upper_value + 1):`

كرّر الجسم المزاح للداخل لكل قيمة في range بالترتيب؛ قيمة stop نفسها لا تدخل.

23. `if is_prime(i):`

اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة True.

24. `print(i, end=" ")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

25. `print()`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

تشغيل فعلي

المدخلات، ثم stdout كما سجّله Python. رسائل input قد تظهر على نفس السطر لأن التسجيل لا يكرر ما كتبته لوحة المفاتيح.

المدخلات

```
10
20
```

stdout

```
Enter the lower value of the range: Enter the upper value of the range: Prime numbers between 10 and
20 are: 11 13 17 19
```

نصائح وأخطاء شائعة

- lower > upper حالة إدخال غير صالحة، لا تبدّل الطرفين بصمت.
- المرجع يحسم 2 مبكرًا، ويرفض بقية الأعداد الزوجية، ثم يجرب القواسم الفردية بخطوة 2.
- لو المجال لا يحتوي عددًا أوليًا، عدم طباعة أعداد نتيجة صحيحة، وليس عطلًا في الحلقة.

اختبر فهمك

ليه upper + 1؟

▼ اعرض السبب

لأن range تستبعد stop، والمرجع يشمل الحد الأعلى نفسه.

لما تقارن بالمرجع: يشمل الطرفين كما في المرجع، مع نفس التحسين الخاص بالأعداد الزوجية والفردية.

06 المضروب: حاصل ضرب يبدأ من 1

(6) Write a Python program, which prompt the user to enter an integer and use a function to calculate the factorial of that integer.

نص الدكتور الأصلي — اضغط الصورة للتكبير.

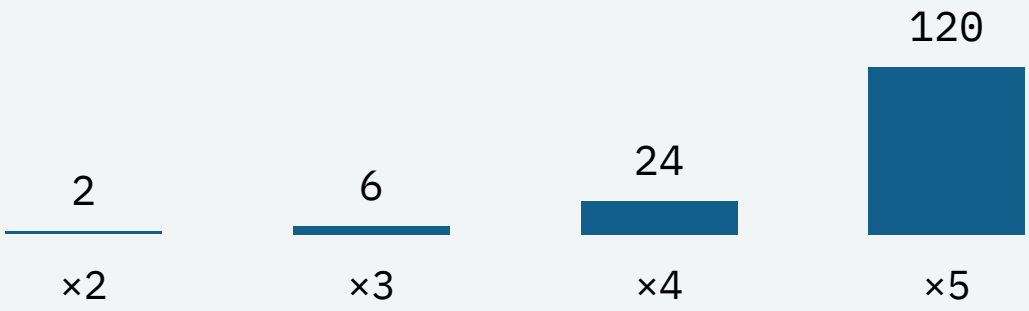
المطلوب إيه؟

احسب n! لعدد صحيح غير سالب. مثلًا 5! هو حاصل ضرب الأعداد من 1 إلى 5، و0! يساوي 1.

الفكرة قبل الكود

المتغير factorial يحتفظ بحاصل ضرب ما مررنا عليه حتى الآن. نبدأ من 1 لأن الضرب في 1 لا يغيّر القيمة. البداية بصفر تجعل كل النتائج صفرًا.

```
start = 1
```



نتتبع مثالًا

5!: يبدأ factorial = 1؛ عند i = 2 يصبح 2؛ عند 3 يصبح 6؛ عند 4 يصبح 24؛ عند 5 يصبح 120، ثم ترجع الدالة 120.

Python اللي السؤال محتاجها

مجال المدخلات؛ or؛ for من 2 إلى n شاملًا؛ *=؛ return خارج الحلقة.

الحل، سطرًا بسطر

اقرأ تعريف الدالة أولًا، ثم ابدأ التنفيذ من البرنامج الرئيسي. عند الاستدعاء ارجع لجسم الدالة، وعند return عد لمكان الاستدعاء.

```python
# Sheet_05, Q_06
# Function to calculate the factorial of an integer
def calculate_factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        factorial = 1
        for i in range(2, n + 1):
            factorial *= i
        return factorial

# Main program
number = int(input("Enter a non-negative integer: "))
if number < 0:
    print("Error: factorial needs a non-negative integer.")
else:
    result = calculate_factorial(number)
    print(f"The factorial of {number} is: {result}")
```

▼ شرح السطور واحدة واحدة

1. `# Sheet_05, Q_06`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

2. `# Function to calculate the factorial of an integer`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

3. `def calculate_factorial(n):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

4. `if n == 0 or n == 1:`

   اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة True.

5. `return 1`

   أعد قيمة 1 إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

6. `else:`

   هذا الفرع يعمل عندما لا يتحقق شرط if المقابل.

7. `factorial = 1`

   احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر return.

8. `for i in range(2, n + 1):`

   كرّر الجسم المزاح للداخل لكل قيمة في range بالترتيب؛ قيمة stop نفسها لا تدخل.

9. `factorial *= i`

   اضرب القيمة الحالية في العامل على اليمين، واحفظ حاصل الضرب الجديد.

10. `return factorial`

أعد قيمة factorial إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

11. `# Main program`

تعليق يشرح الغرض؛ Python لا ينفّذه.

12. `number = int(input("Enter a non-negative integer: "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (int أوfloat)، وخزّن النتيجة في الاسم على اليسار.

13. `if number < 0:`

اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة True.

14. `print("Error: factorial needs a non-negative integer.")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

15. `else:`

هذا الفرع يعمل عندما لا يتحقق شرط if المقابل.

16. `result = calculate_factorial(number)`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر return.

17. `print(f"The factorial of {number} is: {result}")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

تشغيل فعلي

المدخلات، ثم stdout كما سجّله Python. رسائل input قد تظهر على نفس السطر لأن التسجيل لا يكرر ما كتبته لوحة المفاتيح.

المدخلات

```
5
```

stdout

```
Enter a non-negative integer: The factorial of 5 is: 120
```

نصائح وأخطاء شائعة

- n! رمز رياضي؛ في Python تكتب حلقة أو تستعمل math.factorial، ولا تكتب n!.
- 0 و1 يرجعان 1. لا تعتبر -3 حالة مشابهة لهما.
- ضع *= داخل الحلقة وreturn بعدها، حتى تكتمل كل عمليات الضرب.

اختبر فهمك

لو بدأ factorial = 0، هل الحلقة تصل إلى 120؟

▼ اعرض السبب

لا؛ صفر مضروب في أي عدد يظل صفرًا.

لما تقارن بالمرجع: أضفنا رفض العدد السالب في البرنامج الرئيسي. المرجع لا يتحقق منه، ولذلك يرجع 1 عند -3 نتيجةً للحلقة الفارغة؛ هذه ليست قيمة صحيحة للمضروب في مجال السؤال.

07 AP: حد يزيد بمقدار ثابت

(7) Write a Python program, using function, to find the sum of n-terms of Arithmetic Progression Series. This series is expressed as;

$$a\,.\,a+d.\,a+2d\,.\,a+3d\,.\;\;.\;.\;.$$

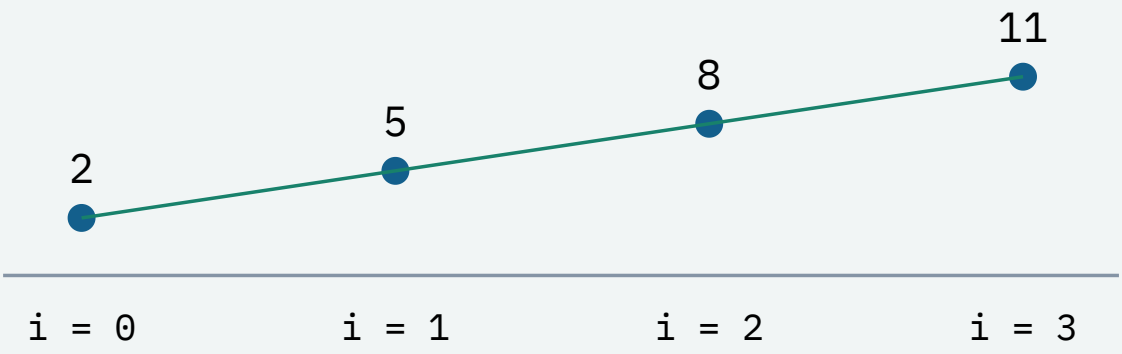
نص الدكتور الأصلي — اضغط الصورة للتكبير.

المطلوب إيه؟

اقرأ أول حد a والفرق الثابت d وعدد الحدود n، واجمع أول n حدود. المطلوب مجموع محدود، وليس طباعة حد واحد.

الفكرة قبل الكود

أول حد يساوي a؛ بعد كل خطوة نضيف d. باستخدام فهرس يبدأ من صفر، الحد رقم i في التتبع هو a + i*d. range(n) تعطي n قيم من 0 إلى n-1.



i	a + i*d	total
0	2	2
1	5	7
2	8	15
3	11	26

نتتبع مثالًا

a = 2، d = 3، n = 4: i = 0، 1، 2، 3؛ الحدود 2، 5، 8، 11؛ المجاميع 2، 7، 15، 26.

Python اللي السؤال محتاجها

function بثلاثة parameters؛ range(n)؛ ضرب قبل جمع؛ total = 0؛ +=.

الحل، سطرًا بسطر

اقرأ تعريف الدالة أولًا، ثم ابدأ التنفيذ من البرنامج الرئيسي. عند الاستدعاء ارجع لجسم الدالة، وعند return عد لمكان الاستدعاء.

```
1  # Sheet_05, Q_07
2  # Function to calculate the sum of an AP series
3  def ap_sum(a, d, n):
4      total = 0
5      for i in range(n):
6          total += a + i * d
7      return total
8
9  # Main program
10 a = float(input("Enter the first term (a): "))
11 d = float(input("Enter the common difference (d): "))
12 n = int(input("Enter the number of terms (n): "))
13
14 if n < 1:
15     print("Error: number of terms must be at least 1.")
16 else:
17     # Calculate and display the sum
18     result = ap_sum(a, d, n)
19     print(f"Sum of the AP series = {result}")
```

▼ شرح السطور واحدة واحدة

1. `# Sheet_05, Q_07`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

2. `# Function to calculate the sum of an AP series`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

3. `def ap_sum(a, d, n):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

4. `total = 0`

   احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

5. `for i in range(n):`

   كرّر الجسم المزاح للداخل لكل قيمة في range بالترتيب؛ قيمة stop نفسها لا تدخل.

6. `total += a + i * d`

   أضف قيمة الطرف اليمين إلى القيمة الحالية للمتغير؛ التحديث يظل محفوظًا للدورة التالية.

7. `return total`

أعد قيمة total إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

8. `# Main program`

تعليق يشرح الغرض؛ Python لا ينفّذه.

9. `a = float(input("Enter the first term (a): "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (floatأو int)، وخزّن النتيجة في الاسم على اليسار.

10. `d = float(input("Enter the common difference (d): "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (floatأو int)، وخزّن النتيجة في الاسم على اليسار.

11. `n = int(input("Enter the number of terms (n): "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (floatأو int)، وخزّن النتيجة في الاسم على اليسار.

12. `if n < 1:`

اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة .True

13. `print("Error: number of terms must be at least 1.")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

14. `else:`

هذا الفرع يعمل عندما لا يتحقق شرط if المقابل.

15. `# Calculate and display the sum`

تعليق يشرح الغرض؛ Python لا ينفّذه.

16. `result = ap_sum(a, d, n)`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

17. `print(f"Sum of the AP series = {result}")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

تشغيل فعلي

المدخلات، ثم stdout كما سجّله .Python رسائل input قد تظهر على نفس السطر لأن التسجيل لا يكرر ما كتبته لوحة المفاتيح.

المدخلات

```
2
3
4
```

stdout

```
Enter the first term (a): Enter the common difference (d): Enter the number of terms (n): Sum of the
AP series = 26.0
```

نصائح وأخطاء شائعة

- ابدأ i من 0 حتى تحسب a نفسها؛ لو بدأت من 1 ستحذف أول حد وتضيف حدًا زائدًا.
- d قد يكون سالبًا أو صفرًا. لا تفترض أن المتتابعة تصعد دائمًا.
- الرموز النقطية في الشيت تفصل الحدود؛ الجمع المطلوب في النص يجمع هذه الحدود.

اختبر فهمك

هل n = 4 يعني آخر i يساوي 4؟

▼ اعرض السبب

لا؛ نحتاج 4 حدود، والفهارس 0 و1 و2 و3.

لما تقارن بالمرجع: نفس جمع المرجع بالحلقة. أضفنا شرط n >= 1 كتحديد واضح لمجال هذا التدريب؛ المجموع الفارغ n = 0 يمكن تعريفه رياضيًا لكنه ليس المثال التعليمي هنا.

08 GP: كل حد مضروب في نسبة ثابتة

(8) Write a Python program, using function, to find the sum of geometric progression series. This series is expressed as;

$$a\ .ar.\ ar^2\ .ar^3\ .\quad ...$$

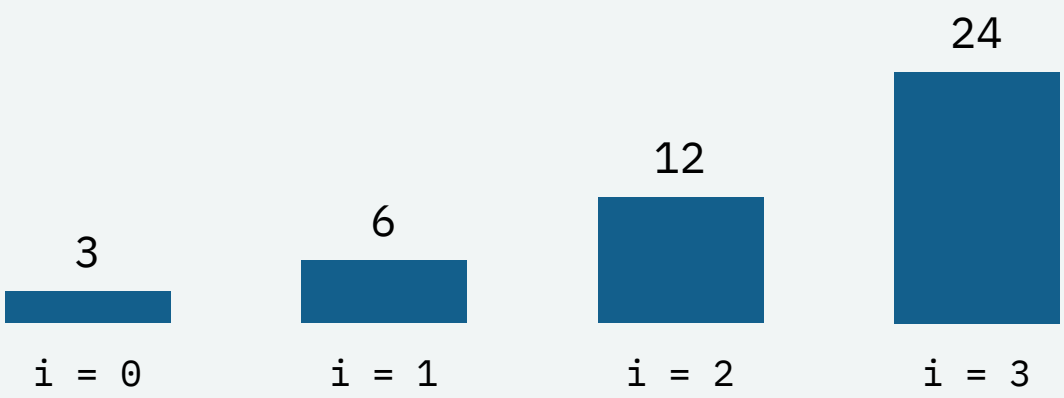
نص الدكتور الأصلي — اضغط الصورة للتكبير.

المطلوب إيه؟

اقرأ أول حد a والنسبة r وعدد الحدود n، ثم اجمع أول n حدود. الشيت لا يذكر n صراحة في سطر المطلوب، لكن كود الدكتور يحدّد مجموعًا محدودًا بعدد حدود يُدخل من المستخدم.

الفكرة قبل الكود

AP تزيد بالجمع؛ GP تتغير بالضرب. الحد ذو الفهرس i هو a * r**i. عند i = 0، العامل r**0 يساوي 1 في هذه الصيغة، فيكون أول حد a.



i	a * r**i	total
0	3	3
1	6	9
2	12	21
3	24	45

نتتبع مثالًا

a = 3، r = 2، n = 4: الحدود 3، 6، 12، 24؛ المجاميع 3، 9، 21، 45. عند a = 3، r = 1، n = 4 يصبح المجموع 12.

Python اللي السؤال محتاجها

الأس **؛ الأقواس؛ range(n)؛ الفرق بين common difference وcommon ratio.

الحل، سطرًا بسطر

اقرأ تعريف الدالة أولًا، ثم ابدأ التنفيذ من البرنامج الرئيسي. عند الاستدعاء ارجع لجسم الدالة، وعند return عد لمكان الاستدعاء.

```
1  # Sheet_05, Q_08
2  # Function to calculate the sum of a GP series
3  def gp_sum(a, r, n):
4      total = 0
5      for i in range(n):
6          total += a * (r ** i)
7      return total
8
9  # Main program
10 a = float(input("Enter the first term (a): "))
11 r = float(input("Enter the common ratio (r): "))
12 n = int(input("Enter the number of terms (n): "))
13
14 if n < 1:
15     print("Error: number of terms must be at least 1.")
16 else:
17     # Calculate and display the sum
18     result = gp_sum(a, r, n)
19     print(f"Sum of the GP series = {result}")
```

▼ شرح السطور واحدة واحدة

1. `# Sheet_05, Q_08`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

2. `# Function to calculate the sum of a GP series`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

3. `def gp_sum(a, r, n):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

4. `total = 0`

   احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

5. `for i in range(n):`

   كرّر الجسم المزاح للداخل لكل قيمة في range بالترتيب؛ قيمة stop نفسها لا تدخل.

6. `total += a * (r ** i)`

   أضف قيمة الطرف اليمين إلى القيمة الحالية للمتغير؛ التحديث يظل محفوظًا للدورة التالية.

7. `return total`

أعد قيمة total إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

8. `# Main program`

تعليق يشرح الغرض؛ Python لا ينفّذه.

9. `a = float(input("Enter the first term (a): "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (floatأو int)، وخزّن النتيجة في الاسم على اليسار.

10. `r = float(input("Enter the common ratio (r): "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (floatأو int)، وخزّن النتيجة في الاسم على اليسار.

11. `n = int(input("Enter the number of terms (n): "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (floatأو int)، وخزّن النتيجة في الاسم على اليسار.

12. `if n < 1:`

اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة .True

13. `print("Error: number of terms must be at least 1.")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

14. `else:`

هذا الفرع يعمل عندما لا يتحقق شرط if المقابل.

15. `# Calculate and display the sum`

تعليق يشرح الغرض؛ Python لا ينفّذه.

16. `result = gp_sum(a, r, n)`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

17. `print(f"Sum of the GP series = {result}")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

تشغيل فعلي

المدخلات، ثم stdout كما سجّله .Python رسائل input قد تظهر على نفس السطر لأن التسجيل لا يكرر ما كتبته لوحة المفاتيح.

المدخلات

```
3
2
4
```

stdout

```
Enter the first term (a): Enter the common ratio (r): Enter the number of terms (n): Sum of the GP
series = 45.0
```

نصائح وأخطاء شائعة

- لا تستخدم a + i*r؛ هذا يصف زيادة ثابتة ولا يصف GP.
- r = 1 يجعل كل الحدود a، والحل بالحلقة يعمل بلا قسمة على صفر.
- هذه متسلسلة محدودة بعدد n. لا تستخدم قانون مجموع لا نهائي ولا تشترط |r| < 1.

اختبر فهمك

ما الفرق بين a + i*d وa * r**i؟

▼ اعرض السبب

الأولى تولّد AP بفروق ثابتة، والثانية GP بنسب ثابتة.

لما تقارن بالمرجع: اعتمدنا تفسير المرجع: مجموع أول n حدود. أضفنا رفض n < 1؛ لم نضف شرطًا على r لأن الجمع المحدود يقبل 1 و0 والنسب السالبة.

09 خانات العدد: خُد الأخيرة واحذفها

(9) Write a Python program, using a function, to generate a random 4-digit number and find the sum of the digits of that number.

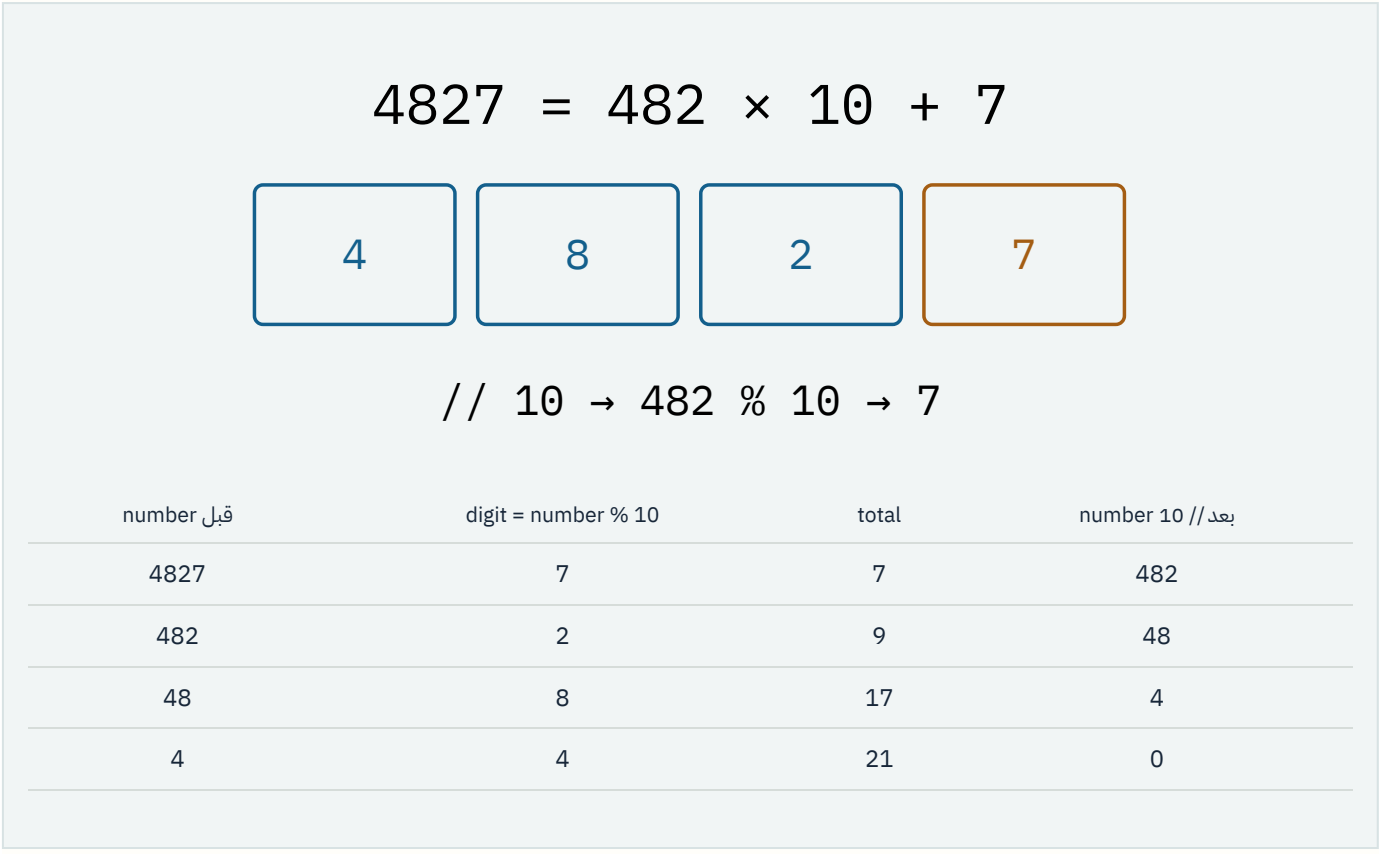
نص الدكتور الأصلي — اضغط الصورة للتكبير.

المطلوب إيه؟

ولّد عددًا صحيحًا من أربع خانات، ثم احسب مجموع خاناته. مثلًا 4827 يُفك إلى 4 و8 و2 و7؛ المثال ثابت للتتبع، بينما التشغيل العادي عشوائي.

الفكرة قبل الكود

للأعداد الموجبة، number % 10 يعطي آخر خانة، وnumber // 10 يحذفها. كرّر الاستخراج والجمع والحذف حتى يصل العدد المحلي إلى صفر.



number قبل	digit = number % 10	total	number 10 // بعد
4827	7	7	482
482	2	9	48
48	8	17	4
4	4	21	0

نتتبع مثالًا

4827 ← خانة 7 ومجموع 7 وباقي عدد 482؛ ثم 2 ومجموع 9 وباقي 48؛ ثم 8 ومجموع 17 وباقي 4؛ ثم 4 ومجموع 21 وباقي 0.

Python اللي السؤال محتاجها

import random؛ randint؛ while؛ %؛ //؛ //=؛ قيمة محلية لا تمحو عدد البرنامج الرئيسي.

الحل، سطرًا بسطر

اقرأ تعريف الدالة أولًا، ثم ابدأ التنفيذ من البرنامج الرئيسي. عند الاستدعاء ارجع لجسم الدالة، وعند return عد لمكان الاستدعاء.

```python
# Sheet_05, Q_09
import random
# Function to find the sum of digits of a number
def sum_of_digits(number):
    total = 0
    while number > 0:
        total += number % 10
        number //= 10
    return total

def generate_number():
    return random.randint(1000, 9999)

# Main program
number = generate_number()
print("Random 4-digit number =", number)

# Find and display the sum of its digits
digit_sum = sum_of_digits(number)
print("Sum of digits =", digit_sum)
```

▼ شرح السطور واحدة واحدة

1. `# Sheet_05, Q_09`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

2. `import random`

   أتح دوال المكتبة .random نستخدم اسم المكتبة ثم نقطة ثم اسم الدالة.

3. `# Function to find the sum of digits of a number`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

4. `def sum_of_digits(number):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

5. `total = 0`

   احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

6. `while number > 0:`

   اختبر الشرط قبل كل دورة. لو أصبح False، اخرج من الحلقة.

7. `total += number % 10`

أضف قيمة الطرف اليمين إلى القيمة الحالية للمتغير؛ التحديث يظل محفوظًا للدورة التالية.

8. `number //= 10`

حدّث العدد المحلي بخارج قسمته الصحيحة على 10؛ تُحذف آخر خانة من العدد الموجب.

9. `return total`

أعد قيمة total إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

10. `def generate_number():`

تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

11. `return random.randint(1000, 9999)`

أعد قيمة random.randint(1000, 9999) إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

12. `# Main program`

تعليق يشرح الغرض؛ Python لا ينفّذه.

13. `number = generate_number()`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

14. `print("Random 4-digit number =", number)`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

15. `# Find and display the sum of its digits`

تعليق يشرح الغرض؛ Python لا ينفّذه.

16. `digit_sum = sum_of_digits(number)`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

17. `print("Sum of digits =", digit_sum)`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

تشغيل فعلي

المدخلات، ثم stdout كما سجّله .Python رسائل input قد تظهر على نفس السطر لأن التسجيل لا يكرر ما كتبته لوحة المفاتيح.

المدخلات

```
(لا توجد مدخلات من المستخدم)
```

stdout

```
Random 4-digit number = 1444
Sum of digits = 13
```

نصائح وأخطاء شائعة

- randint(1000, 9999) يشمل الطرفين؛ البدء من 0 لا يضمن أربع خانات.
- نستخدم // وليس /، لأننا نحتاج عددًا صحيحًا بعد حذف الخانة.
- number //= 10 هو تقدم الحلقة نحو التوقف؛ نسيانه يكرر نفس الخانة بلا نهاية.

اختبر فهمك

هل 4827 % 10 يحذف الخانة الأخيرة؟

▼ اعرض السبب

لا؛ % يعيد الخانة 7، و// يعطي العدد المتبقي 482.

لما تقارن بالمرجع: المرجع يولّد العدد في main ويضع جمع الخانات في function. أضفنا generate_number كدالة صغيرة حتى يكون التوليد أيضًا باستخدام function حسب الصياغة؛ منطق جمع الخانات كما هو. 4827 مثال تعليمي ثابت وليس وعدًا بنتيجة العشوائية.

10 قوى فردية، ومضروب لكل حد

(10) Write a Python program to calculate and print the sum of the following series. Use a function to calculate the sum of the series.

$$x + \frac{x^3}{3!} + \frac{x^5}{5!} + \ldots + \frac{x^n}{n!}$$

نص الدكتور الأصلي — اضغط الصورة للتكبير.

المطلوب إيه؟

اقرأ x وآخر أُس فردي .n احسب الحدود ذات القوى 1 و3 و5 حتى n، واجمع كل قوة مقسومة على مضروب نفس الأس.

الفكرة قبل الكود

n هنا آخر أُس، وليس عدد الحدود. عند n = 5 توجد ثلاثة حدود فقط. دالة series_sum تستدعي calculate_factorial لكل أس، فتتوقف مؤقتًا حتى يرجع مقام هذا الحد.

$$\underset{\text{i = 1}}{\frac{2}{1}} \qquad \underset{\text{i = 3}}{\frac{8}{6}} \qquad \underset{\text{i = 5}}{\frac{32}{120}}$$

```
n = 5 → 3 terms → 3.600000
```

نتتبع مثالًا

x = 2، n = 5: الحد الأول 2/1 = 2؛ الثاني 8/6؛ الثالث 32/120؛ المجموع 3.6 ويُعرض 3.600000. عدد الحدود 3.

Python اللي السؤال محتاجها

range(1, n + 1, 2)؛ دالة تستدعي دالة؛ /؛ **؛ f-string مع :.f6؛ تحقق من العدد الفردي.

الحل، سطرًا بسطر

اقرأ تعريف الدالة أولًا، ثم ابدأ التنفيذ من البرنامج الرئيسي. عند الاستدعاء ارجع لجسم الدالة، وعند return عد لمكان الاستدعاء.

```python
# Sheet_05, Q_10
# A function to calculate factorial in each term
def calculate_factorial(k):
    if k == 0 or k == 1:
        return 1
    else:
        factorial = 1
        for i in range(2, k + 1):
            factorial *= i
        return factorial

# A function to calculate the sum of the series
def series_sum(x, n):
    total = 0

    for i in range(1, n + 1, 2):  # Odd powers only
        total += (x ** i) / calculate_factorial(i)

    return total

# Main program
x = float(input("Enter the value of x: "))
n = int(input("Enter the value of n (odd): "))

if n < 1 or n % 2 == 0:
    print("Error: n must be a positive odd exponent.")
else:
    # Calculate and display the sum
    result = series_sum(x, n)
    print(f"Sum of the series = {result:.6f}")
```

▼ شرح السطور واحدة واحدة

1. `# Sheet_05, Q_10`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

2. `# A function to calculate factorial in each term`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

3. `def calculate_factorial(k):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

4. `if k == 0 or k == 1:`

   اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة True.

5. `return 1`

   أعد قيمة 1 إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

6. `else:`

   هذا الفرع يعمل عندما لا يتحقق شرط if المقابل.

7. `factorial = 1`

   احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر return.

8. `for i in range(2, k + 1):`

كرّر الجسم المزاح للداخل لكل قيمة في range بالترتيب؛ قيمة stop نفسها لا تدخل.

9. `factorial *= i`

اضرب القيمة الحالية في العامل على اليمين، واحفظ حاصل الضرب الجديد.

10. `return factorial`

أعد قيمة factorial إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

11. `# A function to calculate the sum of the series`

تعليق يشرح الغرض؛ Python لا ينفّذه.

12. `def series_sum(x, n):`

تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

13. `total = 0`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر return.

14. `for i in range(1, n + 1, 2):  # Odd powers only`

كرّر الجسم المزاح للداخل لكل قيمة في range بالترتيب؛ قيمة stop نفسها لا تدخل.

15. `total += (x ** i) / calculate_factorial(i)`

أضف قيمة الطرف اليمين إلى القيمة الحالية للمتغير؛ التحديث يظل محفوظًا للدورة التالية.

16. `return total`

أعد قيمة total إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

17. `# Main program`

تعليق يشرح الغرض؛ Python لا ينفّذه.

18. `x = float(input("Enter the value of x: "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (int أوfloat)، وخزّن النتيجة في الاسم على اليسار.

19. `n = int(input("Enter the value of n (odd): "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (int أوfloat)، وخزّن النتيجة في الاسم على اليسار.

20. `if n < 1 or n % 2 == 0:`

اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة True.

21. `print("Error: n must be a positive odd exponent.")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

22. `else:`

هذا الفرع يعمل عندما لا يتحقق شرط if المقابل.

23. `# Calculate and display the sum`

تعليق يشرح الغرض؛ Python لا ينفّذه.

24. `result = series_sum(x, n)`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر return.

25. `print(f"Sum of the series = {result:.6f}")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

تشغيل فعلي

المدخلات، ثم stdout كما سجّله Python. رسائل input قد تظهر على نفس السطر لأن التسجيل لا يكرر ما كتبته لوحة المفاتيح.

المدخلات

```
2
5
```

stdout

```
Enter the value of x: Enter the value of n (odd): Sum of the series = 3.600000
```

نصائح وأخطاء شائعة

- في البسط x**i، وفي المقام i!، وليس n! لكل الحدود.
- إشارة ! لا تُكتب في Python؛ استدعِ calculate_factorial(i).
- اشترط n موجبًا وفرديًا؛ المرجع يطلب odd في رسالة الإدخال لكنه لا يتحقق منه.
- f6.: يغيّر العرض إلى 6 أرقام بعد العلامة العشرية، ولا يغيّر القيمة المخزنة.

اختبر فهمك

عند n = 5 هل نجمع خمسة حدود؟

▼ اعرض السبب

لا؛ نجمع الأسس 1 و3 و5، أي ثلاثة حدود.

لما تقارن بالمرجع: قراءة n كآخر أس فردي مطابقة للصيغة ولكود الدكتور. أضفنا التحقق من 1 =< n و0 =! 2 % n بدل الاكتفاء برسالة الإدخال.

11 Harmonic: عرض الحدود منفصل عن جمعها

(11) Write a Python program to display the n terms of a harmonic series and calculate their sum using functions. The sum of first m terms of harmonic series is expressed as follows;

$$\sum_{n=1}^{n=m} \frac{1}{n} = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \ldots + \frac{1}{m}$$

نص الدكتور الأصلي — اضغط الصورة للتكبير.

المطلوب إيه؟

هناك نتيجتان: عرض شكل الحدود على الشاشة، وحساب قيمتها العددية. نستخدم دالة للعرض وأخرى للجمع، كما في المرجع.

الفكرة قبل الكود

النص "1/3" الذي تطبعه الدالة ليس عملية قسمة؛ هو شكل مقروء للحد. الحساب الحقيقي 1.0 / i داخل دالة الجمع. عدد الحدود هو n في الكود؛ الرمز m في صيغة الشيت يؤدي نفس الدور.

i	النص المعروض	القيمة المضافة	المجموع إلى الآن
1	1/1	1.000000	1.000000
2	1/2	0.500000	1.500000
3	1/3	0.333333	1.833333
4	1/4	0.250000	2.083333

القيم في الجدول معروضة إلى 6 أرقام بعد العلامة العشرية؛ الحساب يجمع القيم قبل تنسيقها.

نتتبع مثالًا

n = 4: النص 1/1 + 1/2 + 1/3 + 1/4. المجاميع بعد الحدود: 1، 1.5، 1.833333...، 2.083333...؛ العرض النهائي 2.083333.

Python اللي السؤال محتاجها

print مع end=""؛ f-string؛ شرط i < n لمنع + أخيرة؛ حلقة تبدأ من 1؛ return مقابل None.

الحل، سطرًا بسطر

اقرأ تعريف الدالة أولًا، ثم ابدأ التنفيذ من البرنامج الرئيسي. عند الاستدعاء ارجع لجسم الدالة، وعند return عد لمكان الاستدعاء.

```python
# Sheet_05, Q_11
# Function to display the terms of the harmonic series
def display_harmonic_series(n):
    print(f"\nHarmonic Series up to {n} terms:")
    for i in range(1, n + 1):
        print(f"1/{i}", end="")
        if i < n:
            print(" + ", end="")
    print()

# Function to calculate the sum of the harmonic series
def calculate_harmonic_sum(n):
    total = 0.0
    for i in range(1, n + 1):
        total += 1.0 / i
    return total

# Main program
num_terms = int(input("Enter the number of terms (n) for the harmonic series: "))
if num_terms < 1:
    print("Error: number of terms must be at least 1.")
else:
    display_harmonic_series(num_terms)

    # Call the function to calculate the sum and display it
    harmonic_sum = calculate_harmonic_sum(num_terms)
    print(f"\nSum of the Harmonic Series up to {num_terms} terms: {harmonic_sum:.6f}")
```

▼ شرح السطور واحدة واحدة

1. `# Sheet_05, Q_11`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

2. `# Function to display the terms of the harmonic series`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

3. `def display_harmonic_series(n):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

4. `print(f"\nHarmonic Series up to {n} terms:")`

   اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

5. `for i in range(1, n + 1):`

   كرّر الجسم المزاح للداخل لكل قيمة في range بالترتيب؛ قيمة stop نفسها لا تدخل.

6. `print(f"1/{i}", end="")`

   اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

7. `if i < n:`

   اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة True.

8. `print(" + ", end="")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

9. `print()`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

10. `# Function to calculate the sum of the harmonic series`

تعليق يشرح الغرض؛ Python لا ينفّذه.

11. `def calculate_harmonic_sum(n):`

تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

12. `total = 0.0`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

13. `for i in range(1, n + 1):`

كرّر الجسم المزاح للداخل لكل قيمة في range بالترتيب؛ قيمة stop نفسها لا تدخل.

14. `total += 1.0 / i`

أضف قيمة الطرف اليمين إلى القيمة الحالية للمتغير؛ التحديث يظل محفوظًا للدورة التالية.

15. `return total`

أعد قيمة total إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

16. `# Main program`

تعليق يشرح الغرض؛ Python لا ينفّذه.

17. `num_terms = int(input("Enter the number of terms (n) for the harmonic series: "))`

اقرأ نص المستخدم ثم حوّله بالنوع المكتوب (int أوfloat)، وخزّن النتيجة في الاسم على اليسار.

18. `if num_terms < 1:`

اختبر هذا الشرط. نفّذ السطور المزاحة تحته فقط عندما تكون النتيجة .True

19. `print("Error: number of terms must be at least 1.")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

20. `else:`

هذا الفرع يعمل عندما لا يتحقق شرط if المقابل.

21. `display_harmonic_series(num_terms)`

استدعاء دالة لتنفيذ مهمتها. تابع جسم الدالة ثم ارجع للسطر التالي هنا.

22. `# Call the function to calculate the sum and display it`

تعليق يشرح الغرض؛ Python لا ينفّذه.

23. `harmonic_sum = calculate_harmonic_sum(num_terms)`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر .return

24. `print(f"\nSum of the Harmonic Series up to {num_terms} terms: {harmonic_sum:.6f}")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

تشغيل فعلي

المدخلات، ثم stdout كما سجّله .Python رسائل input قد تظهر على نفس السطر لأن التسجيل لا يكرر ما كتبته لوحة المفاتيح.

المدخلات

```
4
```

stdout

```
Enter the number of terms (n) for the harmonic series:
Harmonic Series up to 4 terms:
1/1 + 1/2 + 1/3 + 1/4

Sum of the Harmonic Series up to 4 terms: 2.083333
```

نصائح وأخطاء شائعة

- لو بدأت من i = 0، ستقسم على صفر.
- دالة العرض لا تكتب return لقيمة، لذلك ترجع None ضمنيًا. لا تطبع قيمة استدعائها كأنها المجموع.
- استعمل / لا //؛ القسمة الصحيحة ستُسقط الكسور.
- الحلقة التي تحسب المجموع مستقلة عن الحلقة التي تنسّق النص.

اختبر فهمك

هل print("1/2") يحسب نصفًا؟

▼ اعرض السبب

لا؛ يطبع حروفًا. الحساب 1 / 2 ينتج القيمة 0.5.

لما تقارن بالمرجع: احتفظنا بدالتي المرجع وبالتسمية n في الكود. أضفنا رفض n < 1، ووضحنا اختلاف الرمز بين سطر المطلوب والصيغة.

12 Default arguments: القيم المحذوفة لها بديل

(12) Write a Python function with four arguments initialized with zero as default values for these arguments. This function is called from the main program to calculate the sum of arguments with assigned values in the calling statement.

نص الدكتور الأصلي — اضغط الصورة للتكبير.

المطلوب إيه؟

عرّف دالة تجمع أربع قيم، وكل parameter قيمته الافتراضية صفر. جرّب استدعاء الدالة بأربع قيم، وبعض القيم، وقيمة واحدة، وبدون قيم.

الفكرة قبل الكود

القيمة الافتراضية تُستخدم عندما لا يرسل الاستدعاء argument لهذا parameter. لا تُضاف صفرًا فوق القيمة المرسلة، ولا تُلزم المستدعي بكتابة أربعة arguments كل مرة.

الاستدعاء	arg1	arg2	arg3	arg4	الناتج
calculate_sum(10,20,30,40)	10	20	30	40	100
calculate_sum(5,15)	5	15	0 default	0 default	20
calculate_sum(100)	100	0 default	0 default	0 default	100
calculate_sum()	0 default	0 default	0 default	0 default	0

نتتبع مثالًا

(10,20,30,40) → 100؛ (5,15) → 20؛ (100) → 100؛ () → 0؛ وبالاسم arg3=7 → 7.

Python اللي السؤال محتاجها

def calculate_sum(arg1=0, arg2=0, arg3=0, arg4=0)؛ positional arguments؛ keyword arguments كامتداد بسيط.

الحل، سطرًا بسطر

اقرأ تعريف الدالة أولًا، ثم ابدأ التنفيذ من البرنامج الرئيسي. عند الاستدعاء ارجع لجسم الدالة، وعند return عد لمكان الاستدعاء.

```python
# Sheet_05, Q_12
# Function to calculate the sum of four arguments with default values
def calculate_sum(arg1=0, arg2=0, arg3=0, arg4=0):
    return arg1 + arg2 + arg3 + arg4

# Main program
# Calling the function with all arguments explicitly assigned
sum1 = calculate_sum(10, 20, 30, 40)
print(f"Sum 1 (all arguments assigned): {sum1}")

# Calling the function with some arguments assigned, others using default values
sum2 = calculate_sum(5, 15)
print(f"Sum 2 (first two arguments assigned): {sum2}")

# Calling the function with only one argument assigned
sum3 = calculate_sum(100)
print(f"Sum 3 (first argument assigned): {sum3}")

# Calling the function with no arguments, all using default values
sum4 = calculate_sum()
print(f"Sum 4 (no arguments assigned): {sum4}")
```

▼ شرح السطور واحدة واحدة

1. `# Sheet_05, Q_12`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

2. `# Function to calculate the sum of four arguments with default values`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

3. `def calculate_sum(arg1=0, arg2=0, arg3=0, arg4=0):`

   تعريف الدالة ومدخلاتها. النقطتان تبدأان جسم الدالة، وتنفيذه ينتظر الاستدعاء.

4. `return arg1 + arg2 + arg3 + arg4`

   أعد قيمة arg1 + arg2 + arg3 + arg4 إلى مكان الاستدعاء، ثم أنهِ هذا الاستدعاء.

5. `# Main program`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

6. `# Calling the function with all arguments explicitly assigned`

   تعليق يشرح الغرض؛ Python لا ينفّذه.

7. `sum1 = calculate_sum(10, 20, 30, 40)`

   احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر return.

8. `print(f"Sum 1 (all arguments assigned): {sum1}")`

   اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

9. `# Calling the function with some arguments assigned, others using default values`

تعليق يشرح الغرض؛ Python لا ينفّذه.

10. `sum2 = calculate_sum(5, 15)`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر return.

11. `print(f"Sum 2 (first two arguments assigned): {sum2}")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

12. `# Calling the function with only one argument assigned`

تعليق يشرح الغرض؛ Python لا ينفّذه.

13. `sum3 = calculate_sum(100)`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر return.

14. `print(f"Sum 3 (first argument assigned): {sum3}")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

15. `# Calling the function with no arguments, all using default values`

تعليق يشرح الغرض؛ Python لا ينفّذه.

16. `sum4 = calculate_sum()`

احسب الطرف اليمين أولًا، ثم أسند الناتج إلى الاسم أوالأسماء على اليسار. الاستدعاء هنا ينتظر return.

17. `print(f"Sum 4 (no arguments assigned): {sum4}")`

اعرض القيم على الشاشة. أي استدعاء داخل الأقواس يُحسب أولًا؛ end يتحكم في نهاية الطباعة إن وُجد.

تشغيل فعلي

المدخلات، ثم stdout كما سجّله Python. رسائل input قد تظهر على نفس السطر لأن التسجيل لا يكرر ما كتبته لوحة المفاتيح.

المدخلات

```
(لا توجد مدخلات من المستخدم)
```

stdout

```
Sum 1 (all arguments assigned): 100
Sum 2 (first two arguments assigned): 20
Sum 3 (first argument assigned): 100
Sum 4 (no arguments assigned): 0
```

نصائح وأخطاء شائعة

- calculate_sum(5, 15) يملأ أول مكانين، وتبقى القيمتان الأخريان صفرًا.
- calculate_sum() استدعاء صحيح لأن كل parameters لها defaults.
- يمكن كتابة calculate_sum(arg3=7) لتسمية مكان محدد؛ هذا مثال إضافي مش مطلوب استبدال أمثلة الدكتور به.

اختبر فهمك

في calculate_sum(5, 15)، هل الصفر الافتراضي يغيّر 5 و15؟

▼ اعرض السبب

لا؛ نستخدم القيم المرسلة في أول مكانين، والصفر فقط للمكانين اللذين لم تصلهما قيمة.

لما تقارن بالمرجع: أمثلة الحل الرئيسي كما في المرجع. مثال keyword argument امتداد توضيحي في الشرح فقط.

افتح الكود وجربه على جهازك

في PyCharm افتح مشروعك، ثم اضغط بالزر اليمين على مجلد المشروع واختَر New → Python File. يمكنك أيضًا اختيار File → New ثم Python File، أو `Alt+Insert` من نافذة Project على Windows مع الـkeymap الافتراضي. سمِّ الملف مثل `Q_0501`؛ PyCharm يضيف `.py`.

للحفظ: File → Save All أو `Ctrl+S` على Windows. للتشغيل: اضغط بالزر اليمين داخل الملف واختَر 'Run 'Q_0501''؛ الاختصار الافتراضي لتشغيل الملف من المحرر `Ctrl+Shift+F10`. اكتب المدخلات في نافذة Run واضغط Enter بعد كل قيمة مطلوبة.

ملفات الشيت المرجعية اسمها `Q_0501.py` إلى `Q_0512.py`؛ حافظ على هذه الأسماء ولا تحفظ ملفًا بامتداد `.txt` أو `.py`. نص شيت 5 ورسالة الدكتور المرسلة هنا لم يحددا موعدًا أو رابطًا أو اسم مجلد للتسليم؛ اتبع إعلان التسليم الخاص بالمادة عند صدوره.

Computer Basics Guide: حفظ ملف Python · ضغط مجلد ZIP. الروابط الخارجية تحتاج إنترنت؛ الشرح والكود داخل هذه الحزمة يعملان offline.

الأمثلة والكود اتشغّلوا على Python 3.14.6 على macOS. تشغيل Windows / PyCharm لم يُختبر في هذا العمل.

اقرأ المصدر بجانب الشرح، وجرب حالة جديدة بنفسك. تسجيلات التشغيل هنا لا تنفّذ Python داخل المتصفح.